

Encrypted internet traffic classification using a supervised spiking neural network

Ali Rasteh^c, Florian Delpech^a, Carlos Aguilar-Melchor^a, Romain Zimmer^b, Saeed Bagheri Shouraki^c,
Timothée Masquelier^{b,*}

^a Institut Supérieur de l'Aéronautique et de l'Espace (ISAE-SUPAERO), University of Toulouse, France

^b Cerco UMR 5549, CNRS, Université Toulouse 3, Toulouse, France

^c Artificial Creatures Laboratory, Electrical Engineering Department, Sharif University of Technology, Tehran, Iran

ARTICLE INFO

Article history:

Received 6 April 2021

Revised 24 January 2022

Accepted 18 June 2022

Available online 1 July 2022

Communicated by Zidong Wang

Keywords:

Spiking neural network

Surrogate gradient learning

Internet traffic classification

ABSTRACT

Internet traffic recognition is essential for access providers since it helps them define adapted priorities in order to enhance user experience, e.g., a high priority for an audio conference and a low priority for a file transfer. As internet traffic becomes increasingly encrypted, the main classic traffic recognition technique, payload inspection, is rendered ineffective. Hence this paper uses machine learning techniques looking only at packet size and time of arrival. For the first time, Spiking neural networks (SNNs), which are inspired by biological neurons, were used for this task for two reasons. Firstly, they can recognize time-related data packet features. Secondly, they can be implemented efficiently on neuromorphic hardware. Here we used a simple feedforward SNN, with only one fully connected hidden layer, and trained in a supervised manner using the new method known as Surrogate Gradient Learning. Surprisingly, such a simple SNN reached an accuracy of 95.9% on ISCX datasets, outperforming previous approaches. Besides better accuracy, there is also a significant improvement in simplicity: input size, the number of neurons, trainable parameters are all reduced by one to four orders of magnitude. Next, we analyzed the reasons for this good performance. It turns out that, beyond spatial (i.e., packet size) features, the SNN also exploits temporal ones, mainly the nearly synchronous (i.e., within a 200 ms range) arrival times of packets with specific sizes. Taken together, these results show that SNNs are an excellent fit for encrypted internet traffic classification: they can be more accurate than conventional artificial neural networks (ANN), and they could be implemented efficiently on low-power embedded systems.

© 2022 Elsevier B.V. All rights reserved.

1. Introduction

Today, more than half of the global population uses the Internet network. The amount of data exchanged through it increases each year endlessly, giving people more access to information. Whether sending emails, downloading or uploading files, watching videos on streaming platforms, chatting on social media, etc., all these operations involve devices interacting with computer servers located around the world. To improve the integrity of traffic flows, a protocol stack named TCP-IP is used (see e.g. [1] chap. 5). When data are exchanged via this protocol stack, they are broken down into different packets. Traffic classification consists of associating

traffic flows comprised of data packets, with the categories corresponding to different use cases (e.g., email, chat, file transfer, etc.).

Increasing demands to protect privacy have led to the development of methods to encrypt traffic, and of different ways to navigate on the Internet anonymously, such as The Onion Router (Tor) [2] or Virtual Private Networks (VPN) (see e.g. [3]). These techniques encrypt the whole original packets (payload and headers) into a new encrypted payload, adding a new header to ensure traffic handling in the VPN or Tor network. Thus neither the original payload nor header are available for inspection, and the only features that can be used for classification are the encrypted size of the original packet and the time at which the packet is captured. With only these features, packet inspection is not possible anymore, and statistical or machine learning techniques must be used for traffic classification. The state-of-the-art results for such an approach to traffic classification are based on ANNs.

* Corresponding author.

E-mail addresses: ali.rasteh@ee.sharif.edu (A. Rasteh), timothee.masquelier@cnrs.fr (T. Masquelier).

There are different types of neural networks that are used in different contexts:

- Convolutional Neural Networks (CNNs) [4] are mainly employed for image recognition and have a structure inspired by the primate's visual cortex.
- Recurrent Neural Networks (RNNs) [5] are used for automatic speech or text recognition and take into account the temporal dimension by interconnecting neurons to form cycles in the networks.
- Spiking Neural Networks (SNNs) [6] are aimed at imitating biological neural networks. A neuron fires an electrical impulse called “spike” when its membrane potential reaches a specific value, leading to a signal composed of spikes that propagate onto the subsequent neurons. They are very interesting for recognizing time-related patterns. Although SNNs are recurrent networks, their recursive architecture is the result of their neurons' structure, and in that way, they differ from RNNs, which have recurrent connections between different neurons. Also, as described, they are entirely based on biological observations in contrast to RNNs.

The main challenge of SNNs is that their binary nature prevents the use of classic training methods like back-propagating gradient descent [7–9]. To solve this, most current methods are rate-based, meaning that they only consider the number of spikes inside a temporal window, ignoring their times. This is a severe limitation as different spike trains can have the same number of spikes but present distinct temporal patterns. To solve this problem, different alternatives have been developed. For instance, Neftci et al. 2019 [7] proposed a surrogate gradient algorithm that provides spike compatible backpropagation and preserves the temporal patterns. It consists of approximating the true gradient of the loss function while keeping good optimization properties. While training SNNs is hard; they can extract the temporal features of data naturally as the surrogate gradient approach has preserved it. This feature makes them well-suited for data with temporal properties, precisely like the case of internet traffic.

1.1. Contributions

This paper shows that SNNs lead to very promising results for traffic recognition of encrypted traffic. We beat the state of the art [10] for almost every metric on accuracy, precision, or recall. The average accuracy for Tor traffic rises from 67.8% to 98.6%, and for unencrypted traffic from 85% to 99.4%. For VPN traffic, it rises from 98.4% to 99.8%. The number of errors is thus divided by 8 for VPN traffic and by 20 for Tor and unencrypted traffic.

It is important to note that for VPN traffic, the most challenging situation (browsing versus chat traffic) is not present as the dataset used by [10] has no VPN browsing data. One versus all experiments show that SNNs lead to a very significant improvement with all the problematic categorizations such as File Transfer for Tor (increased from 55.8% to 99.7%), Browsing for Tor (from 90.6% to 99.0%), Browsing for unencrypted (90.6% to 99.4%) and Chat for Tor (89% to 99.1%).

These results are obtained with: simpler inputs (300×300 images versus 1500×1500 images), a simpler network (one hidden layer of 100 neurons versus a six hidden layer LeNet-5 CNN with one million neurons), fewer training parameters (31.5 k versus 300 k), and a higher testing data over training data ratio (20/80 versus 10/90).

Our approach is all but fancy; the merit is due to the superior capabilities of SNNs to detect and exploit time-related features. In Section 5 we study this with some experiments that highlight

that short-term synchronicity detection seems to be an essential feature for traffic categorization, and SNNs naturally excel at this.

1.2. Outline

The rest of this paper is organized as follows. Section 2 gives a brief introduction to related works in the field of internet traffic classification. Section 3 gives a complete review of the proposed model for classification of internet traffic using an SNN and with the help of the supervised surrogate gradient method. In Section 4 and 5 some experiments and corresponding results are given to investigate the power of the proposed model to classify internet traffic data. Conclusions and future work are presented in Section 6.

2. Related works

Mainstream techniques for classifying unencrypted traffic, port-based classification, and deep packet inspection [11], and part of the alternative statistical and machine learning techniques, are unusable for encrypted traffic as they use features that are only available before encryption. Such related work will, therefore, not be described here.

There are two usual classification problems: *traffic categorization* (or characterization), which considers types of traffic such as VoIP, File Transfer, Video, Browsing, Chat, P2P, etc.; and *application identification* which considers given applications such as Kazaa, BitTorrent, Google Hangouts Chat, AIM, WhatsApp Web, etc. In this work, we focus on traffic categorization and leave application categorization for future work.

There is a rich literature on traffic categorization through machine learning methods (statistics based or neural network based). Multiple statistical approaches using features compatible with encryption (such as packet times and sizes) have led to scientific publications: fingerprinting [12–14], Bayes [15–18], k-nearest neighbor [19,20], decision trees [19,21], bag of words [21,22] and cross-entropy similarity [14]. Similarly multiple approaches have been studied using different types of neural networks such as: convolutional neural networks [23–27], recurrent neural networks [26], probabilistic neural networks [28] and stacked auto-encoders [23,25].

Regarding accuracy and precision over encrypted traffic, the state-of-the-art results are the Deep Packet approach [29] from 2018 (published in [25]) and the Flowpic approach [10] from 2019. Both build up over the works of Draper-Gil et al. [19], and Lashkari et al. [30] which set up datasets for traffic categorization, the former containing both VPN and unencrypted traffic, and the latter containing both Tor and unencrypted traffic.

In [25] each packet (header and payload) is converted to a normalized byte sequence and used as an input for a 1-D convolutional neural network and a set of stacked auto-encoders. Both are trained with unencrypted and VPN-encrypted traffic. Surprisingly, the resulting networks can categorize traffic for both unencrypted and VPN traffic without using time-related features (but size). Performance is quite good with 94% average recall and 93% average recall. Unfortunately, for the unencrypted vs. Tor dataset, there is an important drop in average recall (down to 57%) and average precision (down to 44%).

The Flowpic approach [10] uses all spatial and temporal information contained in a traffic flow without looking at its content. Flow packets, encrypted or not, are represented with a 2D-histogram, in the form of a square picture, with packet size in ordinate and normalized arrival times in abscissa. The main idea of Flowpic is that with this approach, the traffic category can be derived from image recognition techniques. The images are thus

fed to a LeNet-5 [31] convolutional neural network, and categorization is done in a class vs. all game with 97% average accuracy for unencrypted traffic, 99.7% for VPN traffic, and 85.7% for Tor traffic. Given their nice results over all three sorts of traffic, Flowpic will be our main point of comparison.

3. Proposed model

3.1. Prospects

The approach used in this paper is very similar to the one employed in [10]. Histograms represent flow packets grouped by size and feed the neural network. The difference relies on using an SNN to operate such classification instead of a CNN. No previous research with that method has been reported, and the results will be mainly compared to the study presented in [10]. To ensure that this comparison is fair, we follow the same dataset generation and processing as in [10].

3.2. Dataset generation

To generate the data, we used two public datasets from the University of New Brunswick (UNB) called “2016 ISCX VPN-nonVPN traffic dataset” from [19] and “2017 ISCX Tor-nonTor dataset” from [30], both used in the closely related works [25,10]. They comprise a range of packet captures (pcap) format files corresponding to traffic flow captures of five main classes (Video, VoIP, Chat, File Transfer, and Browsing) using three different types of encryption (Unencrypted, VPN, and Tor). We converted them into comma-separated values (CSV) files containing arrival times and sizes to exploit the data. We have not considered any restriction about the number of packets or their size. Again it is important to note that there is no Browsing-VPN data in the ISCX dataset, so we have not used this type of data for training or testing our model (as neither did the authors of [19,30,25,10]). Introducing such traffic and making a fair comparison with other approaches when it is included is left as future work.

3.3. Data processing methodology

The first step is to extract the suitable features from these flow packets. A traffic flow can be seen as a sequence of packets with the same source and destination IP. They are bidirectional: forward when a message is sent from source to destination, and backward when a message or acknowledgment (ACK) is sent back from destination to source. Thus, for each exchange, we consider data from both directions of the flow. We pick, as in [10], packet size and time of arrival as features, since they are always available, whether the traffic is encrypted or not. Then, we split each flow every 15s into equal sequences lasting 60s. It would be helpful to increase even the number of samples of the dataset by splitting the flows into smaller sequences, but there are silent periods of a few seconds in the data, so it is impossible to split the data with much less than 15s steps as it leads to samples without any helpful information. We have, therefore, csv files representing one-minute chunks of a communication session and containing packet sizes and times of arrival. As a result, we generate 2D-histograms of packet sizes as a function of the time of arrival from each csv file as shown in Fig. 1. We have two series of histograms for one flow, each corresponding to one direction (forward and backward), concatenated along the y (size) dimension. The histogram values correspond to the number of packets with the same (time size) coordinates. In addition, the resolution of the histograms can be adjusted by increasing or decreasing the number of bins it contains. If the number of bins is too high, the memory and computation requirements

for the classification training become prohibitive. Moreover, as we will associate one input line to each size bin, overfitting can become an issue. Indeed, adding one size bin increases the number of trainable weights by the number of hidden neurons, here 100. If the number of bins is too low, it degrades the histogram quality and, therefore, the network’s ability to recognize the right traffic class. By trial and error, we found that 300 time bins and 300 size bins are a good choice (whereas [10] used 1,500x1,500). Once implemented, the histograms can be classified according to their category. Some examples of images obtained are shown in Fig. 2.

3.4. Labeling dataset

A label is attributed to each csv file according to its traffic category. A label is a number giving both the traffic class (Chat, Browsing, File Transfer, Video or VoIP) and the encryption method (Tor, VPN, or Unencrypted). For instance, label two matches with Video over VPN traffic while label seven matches with File Transfer over Tor traffic. Once created, the csv files containing the histograms are distributed between groups of equal size, called batches, used as input data for the neural network training. Thus, the batch size, representing how many histograms each batch contains, is a parameter to be adjusted. Increasing it avoids scattering data, whereas decreasing it introduces stochasticity, benefiting the gradient descent. Moreover, we use a weighted random sampler on our training dataset. For each class, a weight is attributed to each file, which is inversely proportional to the number of files in this class. This allows to balance the batches in the training dataset and therefore to improve the classification performance.

3.5. The model

3.5.1. Spiking neural network

An SNN can be seen as a type of biologically inspired deep learning model whose state evolves in time and in which spikes transmit information between neurons [7]. In most SNNs, neurons are modeled using the leaky integrate-and-fire (LIF) model. It introduces a neuron dynamic able to capture temporal patterns. In discrete-time, it can be described as follows:

$$V[n+1] = \beta(V[n] - R[n]) + (1 - \beta) \sum_j w_j S_j^{in}[n] \quad (1)$$

$$S^{out}[n+1] = \theta(V[n+1] - V_{th}) \quad (2)$$

$$R[n+1] = V_{th} \cdot S^{out}[n] \quad (3)$$

V represents the membrane potential, n the discrete time, $\beta = e^{-\frac{\Delta t}{\tau}}$ the leak coefficient (with τ the membrane time constant), $\sum_j w_j S_j^{in}$ the weighted sum of input spikes, R the reset function due to the output spikes, S^{out} the output spike, V_{th} the neuron’s threshold voltage, and θ the Heaviside step function. Eq. 2 implies that whenever the membrane potential reaches the threshold, an output spike is fired and the potential is reset. As for ANNs, an SNN can be described as a large number of neurons connected via weighted synapses that are represented by weights. A weight is a number that evaluates how information, here spikes, is transmitted between two neurons. Thus, training a network means adjusting the weights so that data provided as input leads to the activation of the correct output. The difference between the result provided by the network and the desired result is computed by a loss function and has to be minimized. Here, we use the cross-entropy function H calculated as follows

$$H(p, q) = - \sum_i p_i \log(q_i) \quad (4)$$

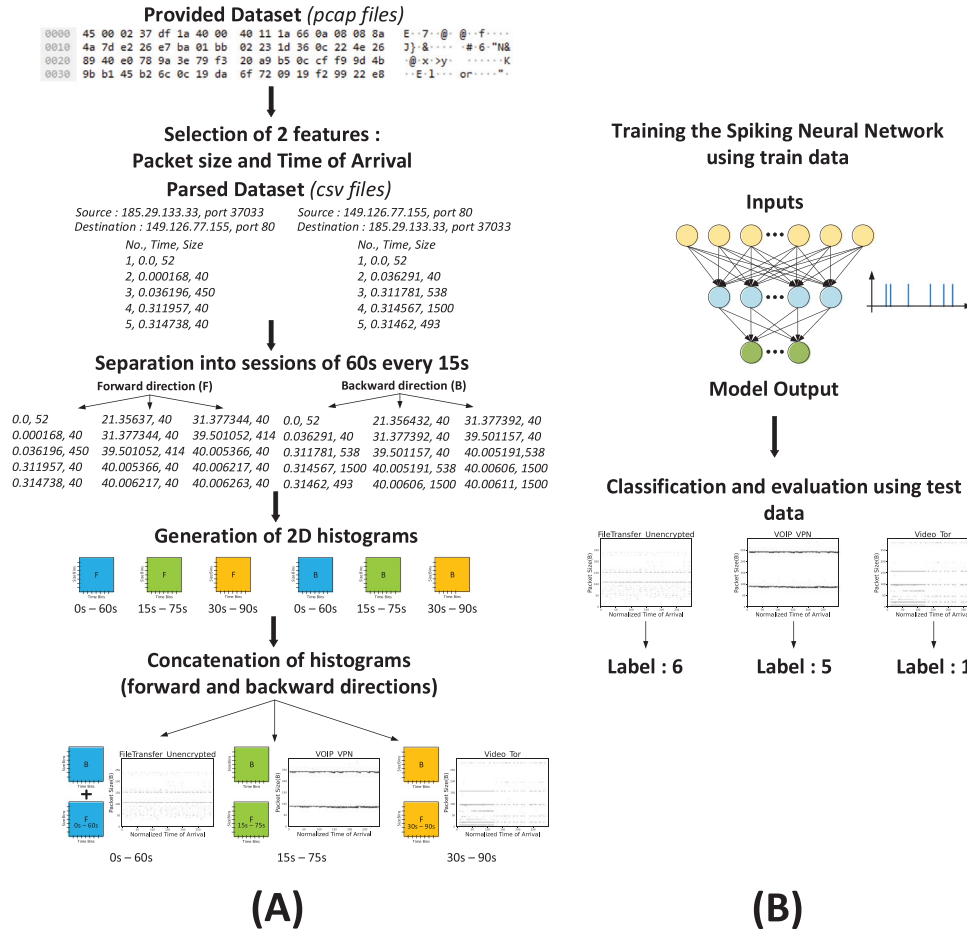


Fig. 1. Architecture proposed for classifying Internet traffic using a SNN. Section (A). This section is the part that is similar to [10]; at first, from the original pcap files in the dataset, we generate csv files containing the time of arrival and the size of each packet in each flow (forward and backward direction). We separate these sessions into sessions of 60s with 15s of gap between them. Next, we generate two-dimensional histograms using these sessions. The time bins are considered on the X-axis and the size bin on the Y-axis. We also concatenate the forward and backward traffic histograms to generate the final histogram as the input of our network. Section (B). This section describes our model. The concatenated histograms are fed to the SNN, and the network is trained using these histograms. Finally, the evaluation of the model is done using test dataset histograms.

With $p_i = 1$ if the true label is i and 0 otherwise, and q_i the predicted value of the model. To adjust the weights and train the network, the mainly used method that has proved to be efficient is backpropagation. It consists of updating the weights from the output layer back to the network's input layer in a way that minimizes the loss function. As SNN are recurrent (see Eq. 2), backpropagation through time (BPTT), which consists in unfolding through time the network before applying backpropagation, is employed (see Fig. 3). Still, this method requires to calculate the gradient of the activation function. SNNs pose an issue because the Heaviside function gradient is 0 everywhere except in 0, which implies that no information can flow through the activation function during the backpropagation. Recent research has managed to solve this problem by switching the true gradient by a surrogate function [7]. Here we used the derivative of a sigmoid since a sigmoid is continuous and can approximate the Heaviside function as seen in Fig. 4.

One of the main assets of SNNs compared to other networks is that inference can be implemented at low power using event-driven computation on neuromorphic chips [32]. They are thus appealing for embedded systems such as satellites. Furthermore, SNNs are adapted to dynamic inputs: they detect synchronicity and other time-related features. The proposed architecture for traffic classification consists of two fully connected layers: one Spiking

Dense Layer made up of 100 neurons with 300 inputs and one Read Out Layer with 14 neurons corresponding to the different categories to classify (Fig. 5). We adapted the s2net code (available at <https://github.com/romainzimmer/s2net>), designed to train both fully connected and convolutional SNNs using surrogate gradient learning, and based on PyTorch. This code has already given excellent results on the Google speech commands dataset [33,34], and for epileptic crisis detection from EEG signal [35]. In our case, the network receives histogram batches as inputs and then delivers a prediction vector over the 14 labels (corresponding to the 14 categories in Fig. 2). The first layer is a dense spiking layer. The neurons' potentials (V) are calculated for the 300 time steps, and spikes are generated according to Eq. 2. The second dense layer (called the Readout layer) can be seen as a spiking layer with an infinite threshold. The potential of each neuron is calculated over the 300 time steps using Eq. 2, and then the neuron with the highest mean potential is the winner and specifies the prediction of the network for the corresponding input.

4. Experiments

Our dataset has a total of 17,450 data samples before being divided into different sets. The dataset has been then split into

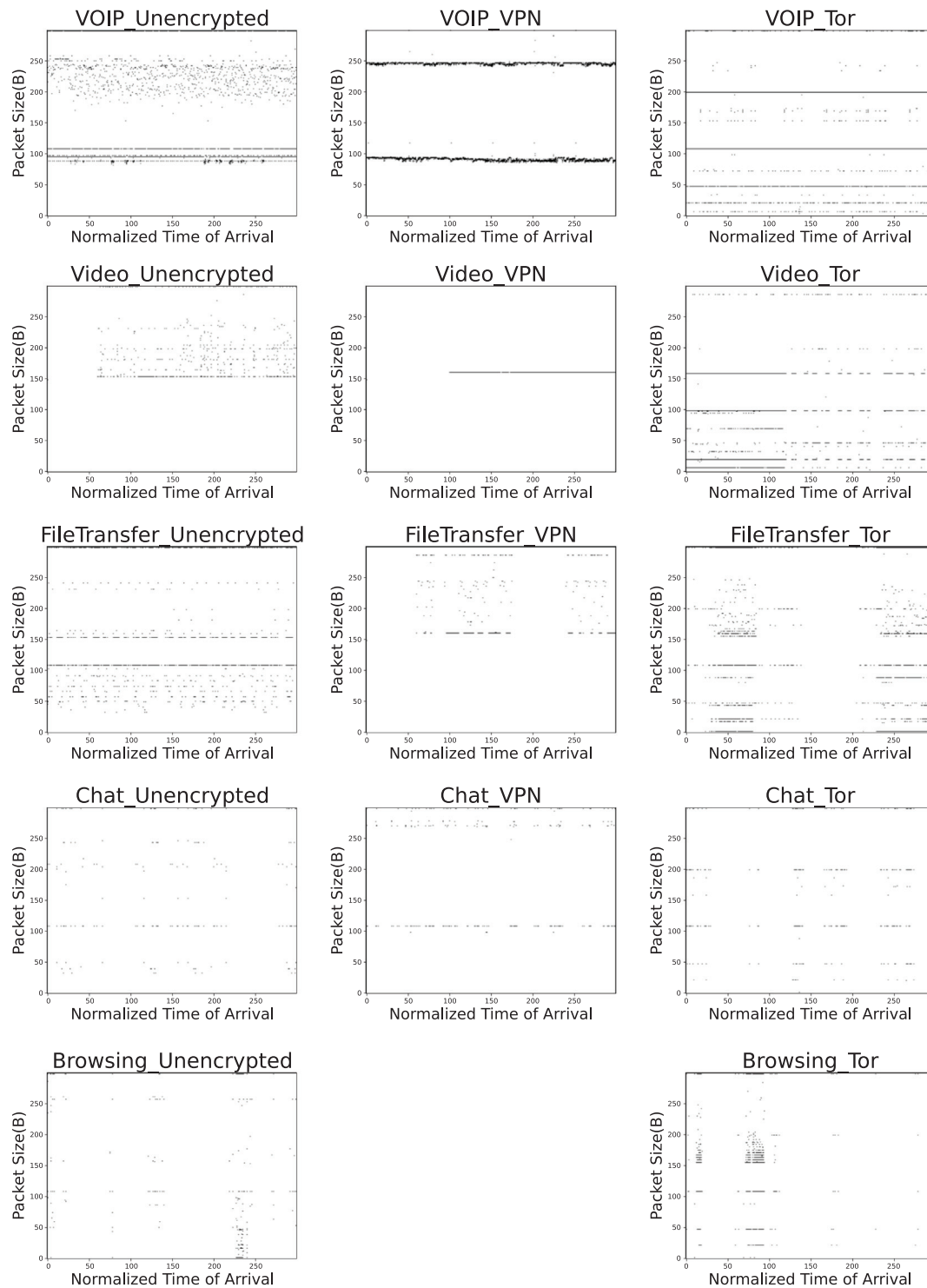


Fig. 2. Examples of obtained two dimensional histograms for each traffic category. The X-axis and Y-axis show time and size bins, respectively. For clarity, all non-empty bins are represented in black, whatever the packet count. Note: there is no Browsing-VPN data in the ISCX dataset.

three separated sets: 65% of samples used to train the model, 15% to validate it during the training phase, and 20% to evaluate it during the testing phase. Before splitting, the number of data samples in different categories is a few hundred for Video/File transfer/Chat in each of Non-VPN/VPN/Tor techniques and a few thousand in the case of VoIP/Browsing. The training phase consists of optimizing the network by adjusting its weights, but also the leak coefficients

β (we had one trainable β per layer, as previous works suggested that training these coefficients can increase accuracy [36,37]). Also, the value of neuron's threshold V_{th} per layer is learned by the gradient descent. The validation set is used to control model performances and tuning hyper-parameters, whereas the testing phase enables the evaluation of final model performance. As a result, we extracted independent data from the dataset used only for

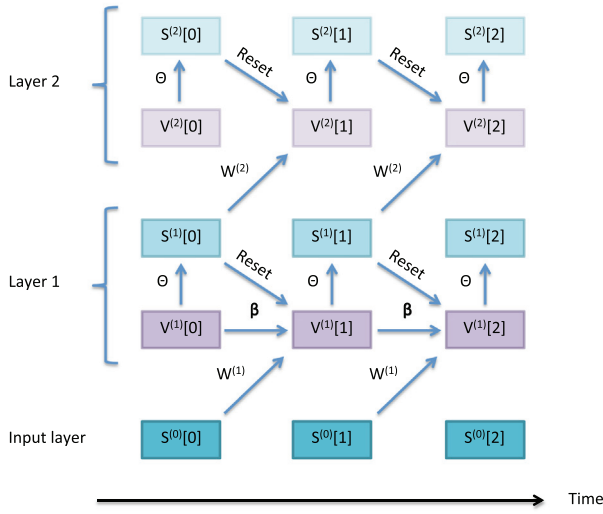


Fig. 3. The unfolded Spiking Neural Network. This image shows the unfolded SNN demonstrating how the gradient is backpropagated through the network using BPTT.[7].

the test. Otherwise, there could be a risk of developing an over-fitted model. This would be an important issue since it would mean the model is biased, too specific, and unable to classify new data correctly. For training, we set the batch size to 128 and used 30 epochs. Cross entropy loss was used as the loss function. We also used the *Adam* optimizer along with *ReduceLROnPlateau* as a scheduler which allows us to adjust the learning rate according to the epoch and thus to improve gradient descent for BPTT (the initial value of learning rate was $5e-4$). To avoid over-fitting, we also introduce a regulatory loss coefficient that penalizes solutions using a large number of spikes. Thus, even if classification may be less efficient, the number of spikes remains quite low, saving energy. This could be very significant in embedded systems, for

instance. In the experiments, we have tested several values for different hyper-parameters of our model, which are explained in Table 1. The mentioned hyper-parameters are tuned by exhaustive search. We have used Nvidia Tesla T4 GPU for final experiments, and the average run time is 131 s for each epoch of training and 19 s for evaluation on the test dataset.

5. Results

As explained in Section 3.4 our model predicts both the traffic category (e.g., VoIP) and the encryption technique (e.g., VPN) at the same time, and it was trained over the whole dataset. This is in contrast with [10], where the dataset was split in three according to the encryption techniques, and different classifiers were trained for each encryption technique and for each task (“one vs. all” or multi-class categorization). The task in [10] is thus simpler as each of their three classifiers only needs to find one among five categories (or four in the VPN case), whereas in our case, it must find one among 14. That being said, we can compute the accuracy of our network for all the tasks used in [10], because they are sub-tasks of the tasks we solved (i.e., all performance measures can be computed from the confusion matrix of Fig. 8). However, it is important to note that not only does the proposed model outperform Flowpic at its sub-tasks, but it also solves a more complicated problem with a very small error rate.

We first evaluated our model in one vs. all classification, meaning that each class is compared to the rest of the dataset. We have used three different performance metrics. For one class, recall (Re) measures the model’s capacity to successfully retrieve the class and not to miss it. Precision (Pr) measures the ability of the model to specifically retrieve the class and ensure the class returned is the good one. Accuracy (Ac) represents a global score quantifying if the model is often right or not.

$$Pr = \frac{TP}{TP + FP} \quad (5)$$

$$Re = \frac{TP}{TP + FN} \quad (6)$$

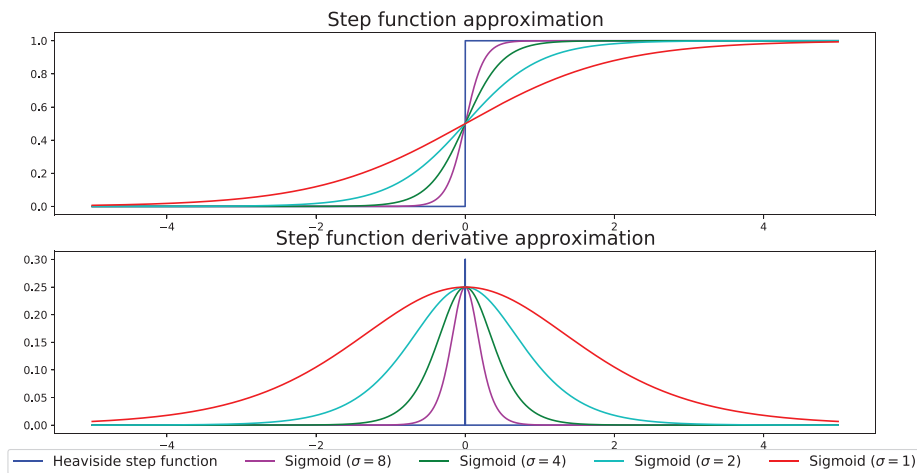


Fig. 4. Surrogate gradient for activation function. Different Sigmoid functions can be a suitable approximation for the Heaviside function. Because the step function is not differentiable and has no gradient, we need to use a surrogate gradient in the backpropagation algorithm of training the SNN, so the sigmoid function is assumed as a substitution of the main Heaviside function. Its gradient is used as a surrogate gradient in the backpropagation algorithm. In the forward path, we use the main Heaviside function, but the surrogate gradient is used in the backward path.[7].

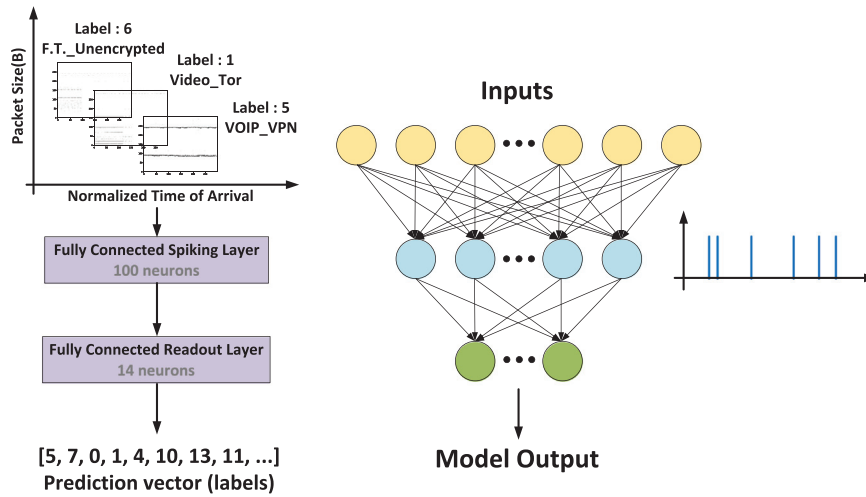


Fig. 5. Architecture of the proposed network. In the proposed architecture, there is just one hidden layer with 100 neurons and one Readout layer with 14 neurons. The size of input histograms fed to the network (in the input layer) is (300,300) which is 300 for time bins and 300 for size bins on the 2D-histogram. The time step of the spiking neurons is set to the time bins dimension, and the input layer size is set to the size bin dimension. At the Readout layer (with 14 neurons), each neuron is representative of one traffic category, and the winner neuron is the neuron with the highest mean potential during all time steps. The winner neuron tells which class is chosen by the network.

Table 1

Range of the hyper-parameters tested in our experiments. The Bold values show the value with the best final test accuracy. The best values are found by exhaustive search.

Hyper-parameter	Range of tested values
Number of dense layer neurons	20, 40, 60, 80, 100 , 120, 140, 160, 200, 250, 300
Number of time bins in histograms and number of time steps of neurons	100, 200, 300 , 400, 500, 750, 1,000, 1,500, 2,000, 2,500
Number of packet size bins in histograms	100, 200, 300 , 400, 600

$$Ac = \frac{TP + TN}{TP + TN + FP + FN} \quad (7)$$

TP, FP, TN, FN stand for True Positive, False Positive, True Negative, and False Negative, respectively. For the sake of brevity, in this section, we report the accuracy measure, but the other measures can be found in A. Table 2 shows the results for each category and encryption technique. It extends Table A.4 in [10] with our

results (we kept the same form to simplify the comparison). Note, however, that in [10] they also provide accuracy when using a classifier (e.g., VPN-trained) on another setting (e.g., Tor). As we only have one classifier for all the categories, we only kept for the comparison the best figures of the original table (which correspond to using the right classifier in its own setting, e.g., the VPN-trained classifier to classify VPN traffic). We have reached at least 99.0% of accuracy in all categories and even 100% for some categories. Moreover, our model outperforms [10]'s model for all categories except File transfer-VPN, in which Flowpic slightly performs better (99.9% vs. 99.8% for us). Figs. 6 and 7 compare again the results of our network with [10] in categorization tasks on different traffic classes and different encryption techniques using error rates instead of success rates. As the accuracy rates are all very close to 100%, it is hard to compare the relative improvements. These graphs are probably a better indicator of the relative multiplicative drop obtained on the number of errors when using our strategy. The proposed model also reached an accuracy of 96.7% on the

Table 2

Comparison between results of this paper and Flowpic [10] in one vs all categorization of different traffic classes. It is noteworthy that the unencrypted traffic is named Non-VPN in Flowpic [10]. As the table depicts, the results are improved in all traffic classes and encryption methods except File transfer-VPN, in which Flowpic performs slightly better.

Class	Accuracy (%)			
VoIP	Work/Data	Unencrypted	VPN	Tor
	Flowpic	99.6	99.9	93.3
	This paper	100.0	100.0	99.4
Video	Work/Data	Unencrypted	VPN	Tor
	Flowpic	99.9	99.9	99.9
	This paper	100.0	100.0	100.0
File Transfer	Work/Data	Unencrypted	VPN	Tor
	Flowpic	98.8	99.9	55.8
	This paper	100.0	99.8	99.7
Chat	Work/Data	Unencrypted	VPN	Tor
	Flowpic	96.2	99.2	89.0
	This paper	99.4	99.8	99.1
Browsing	Work/Data	Unencrypted	VPN	Tor
	Flowpic	90.6	–	90.6
	This paper	99.4	–	99.0

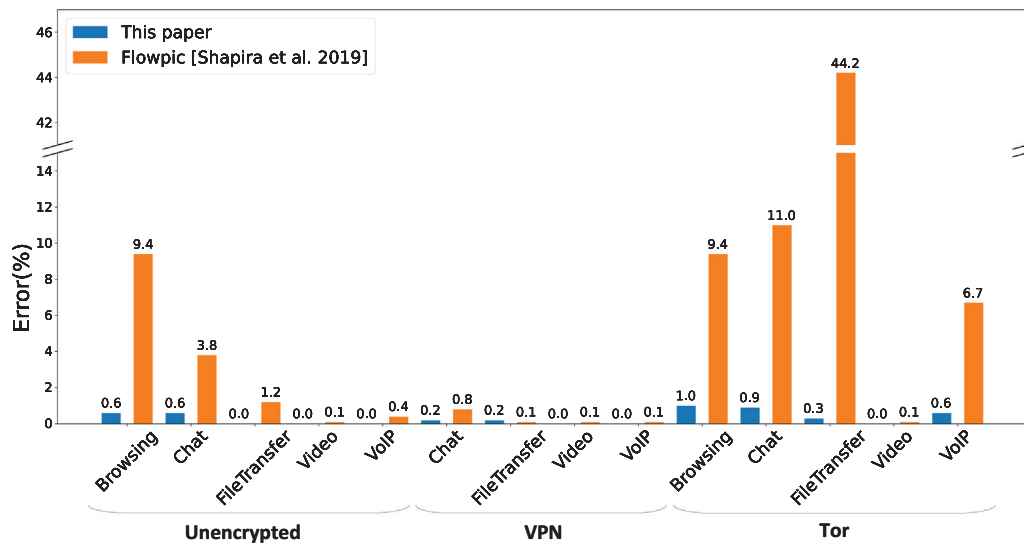


Fig. 6. Comparison of one vs all errors on categorization of different traffic classes and encryption methods between this work and Flowpic [10]. Note that in Flowpic, the unencrypted traffic is named Non-VPN. The task was one versus all classification of different traffic categories with the data from just one encryption method (Unencrypted, VPN, Tor). We have used the error rate on classification (100-accuracy) of the data instead of the accuracy to highlight the relative multiplicative gap between the performance of the proposed network in comparison to Flowpic [10]. As the Figure depicts, this paper is better in all traffic classes and encryption methods except File transfer-VPN which Flowpic performs slightly better.

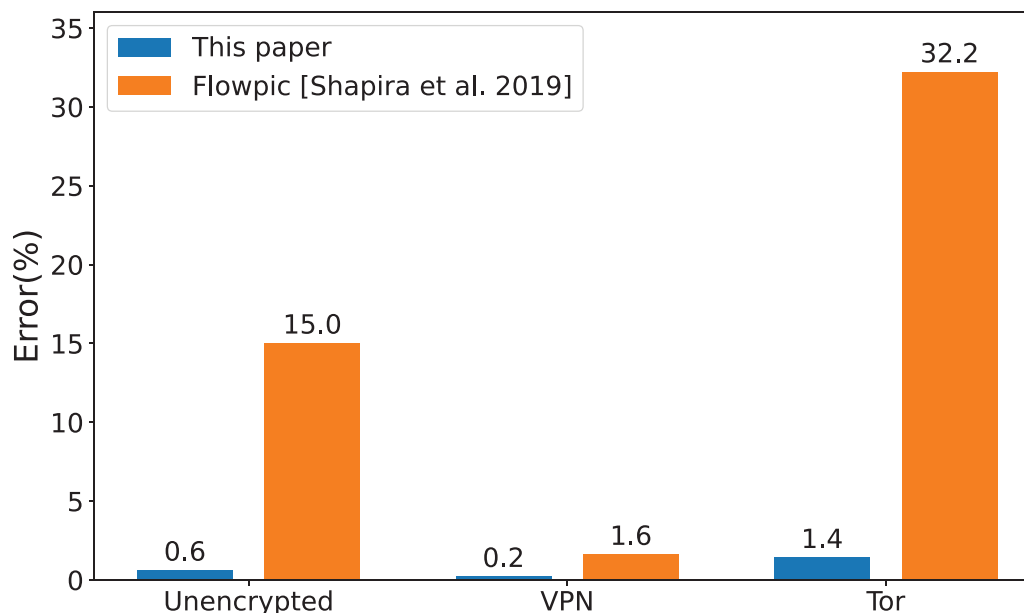


Fig. 7. Comparison of error rates on categorization of different encryption techniques between this work and Flowpic [10]. The task was multi-class categorization between different traffic classes of each encryption method. We outperformed Flowpic for all encryption methods.

multi-class categorization of different encryption techniques, which is better than the 88.4% reported by [10] (see Table A.3, row 7).

Finally, we reached an overall accuracy of 95.9% and average precision of 96.9% (full precision measures are available in A). Note that [10] could not report this global accuracy because they always separated the encryption techniques. Fig. 8 shows the confusion matrix. We observe some confusions between, for instance, Chat Tor and Browsing Tor or between VoIP Unencrypted or VoIP VPN.

When analyzing the failed instances, we observed that most of them contained too few packets to be classified correctly.

The results highlight that our SNN can classify different traffic classes with very good performance according to three encryption techniques. This is even more noticeable considering the limited amount of data needed for training and the simplicity of our network. It is fully connected with only one hidden layer compared to the two convolutional layers plus one fully connected layer used in [10]. This can be very useful for embedded systems such as satellites due to their low energy requirements.

Predicted \ Label	Video Unencrypted	Video Tor	Video VPN	VoIP Unencrypted	VoIP Tor	VoIP VPN	F.T. Unencrypted	F.T. Tor	F.T. VPN	Chat Unencrypted	Chat Tor	Chat VPN	Browsing Unencrypted	Browsing Tor
Video Unencrypted	99.50%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
Video Tor	0.00%	100.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
Video VPN	0.00%	0.00%	100.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
VoIP Unencrypted	0.50%	0.00%	0.00%	84.20%	0.00%	0.30%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
VoIP Tor	0.00%	0.00%	0.00%	0.00%	98.30%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
VoIP VPN	0.00%	0.00%	0.00%	15.80%	0.00%	99.70%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
F.T. Unencrypted	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	100.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
F.T. Tor	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	98.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
F.T. VPN	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	97.90%	0.00%	0.00%	0.00%	0.00%	0.00%
Chat Unencrypted	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	92.50%	0.00%	0.00%	0.80%	0.00%
Chat Tor	0.00%	0.00%	0.00%	0.00%	0.70%	0.00%	0.00%	1.00%	0.00%	0.00%	92.10%	0.00%	0.00%	0.50%
Chat VPN	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	2.10%	0.00%	0.00%	100.00%	0.00%	0.00%
Browsing Unencrypted	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	7.50%	0.00%	0.00%	99.20%	0.00%
Browsing Tor	0.00%	0.00%	0.00%	0.00%	1.00%	0.00%	0.00%	1.00%	0.00%	0.00%	7.90%	0.00%	0.00%	99.50%

Fig. 8. Confusion matrix of the proposed model on the categorization of all classes simultaneously (including all the data from different traffic classes and encryption methods and 14 classes as depicted in the matrix). The classes on the lines are the predicted labels while the classes on the columns are the accurate labels.

Finally, we performed some additional analyses to get more insight into why our SNN reaches such a good accuracy despite its simplicity. Firstly, we tried to destroy all the temporal features at inference time (after regular training). This was done by shuffling the histograms along the temporal dimension. More specifically, we randomly permuted the elements of each row of the histograms, using a different permutation for each row. This completely destroys the temporal features, but not the spatial ones (for example, if a specific sort of traffic tends to use larger packets than another, it can still be used for discrimination). Such an approach led to a total accuracy of only 90.6%, and the comparison to the accuracy in normal mode (95.9%) indicates that our SNN does exploit temporal features.

Secondly, we tried to destroy all the temporal features but the synchronous ones (still at inference time, after regular training). This was done by reshuffling the histograms along the temporal dimension, but this time the same permutation was used for all the rows of a given histogram. This means that if multiple packets with different sizes arrived in the same time bin before the shuffling, they still arrived in the same time bin after it. Conversely, any inter-time-bins temporal information, i.e., long-range temporal correlations, was lost. Of course, this second shuffling method also conserved the spatial features. This second experiment led to an accuracy of 95.1%, almost as good as the baseline one (95.9%). This means that our SNN mainly exploits synchronous features (with a 200 ms resolution), largely ignoring longer-range correlations. Of course, this second sort of shuffling is more harmful when increasing the resolution (e.g., with 40 ms time bins, the accuracy drop is about 2%, indicating that cross-correlations with a 40–200 ms lag matter).

To confirm the range of valuable correlations, we went back to a 200 ms resolution, and we analyzed the final values of the leak coefficients β (recall that it is trainable). It turns out that, whatever the initial values, the β of the hidden layer converges towards ~ 0.42 . With such a fast decay, neurons quickly forget any packet that arrived before the current time bin. In other words, neurons mostly care about synchrony (with a 200 ms resolution). In the last experiment, we forced $\beta = 0$. In this extreme case, the neurons become

stateless: they only care about current inputs. This led to an accuracy of 95.2%. This is consistent with the second shuffling experiment and confirms that inter-time-bins correlations are not very informative when the time-bin size is large enough to capture short-term synchronicity.

Altogether, these additional experiments confirm that the dataset contains temporal features, mostly synchrony patterns (with a 200 ms resolution), that are highly diagnostic and efficiently exploited by an SNN with a fast decay.

6. Conclusion

In this paper, we proposed an approach to use an SNN to classify different traffic classes encrypted with different techniques (via VPN or Tor). No previous work has used SNNs for the purpose of traffic classification. We only considered packet size and time of arrival of flow traffic. Therefore, the input data were 2D-histograms representing a session of 60s in both forward and backward directions. We reached an overall accuracy of 95.93% and an average precision of 96.9% on the test dataset with a simple network made up of only one hidden layer and trained on only 65% of the dataset. Thus, we have shown that SNN can be very relevant for Internet traffic classification with applications in the industry since SNN can run on neuromorphic hardware with low energy consumption [32]. Finally, progress can be made, and the model performance can be enhanced by considering spiking convolutional layers that are very promising.

CRedit authorship contribution statement

Ali Rasteh: Methodology, Software, Validation, Formal analysis, Investigation, Writing - original draft, Visualization. **Florian Delpech:** Methodology, Software, Validation, Formal analysis, Investigation, Writing - original draft, Visualization. **Carlos Aguilar-Melchor:** Conceptualization, Methodology, Formal analysis, Investigation, Writing - review & editing, Supervision, Project adminis-

tration. **Romain Zimmer:** Software. **Saeed Bagheri Shouraki:** Data curation, Writing - review & editing. **Timothée Masquelier:** Conceptualization, Methodology, Formal analysis, Investigation, Data curation, Writing - review & editing, Supervision, Project administration.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

We thank the authors of the Flowpic paper [10], Tal Shapira and Yuval Shavitt, for sharing their processed pcap files with us.

Appendix A. Complete results

In this section, we bring complete performance results, while in Section 5 we just mentioned the accuracy. Complete results of one vs all categorization are presented in Table A.3 for different traffic classes, and in Table A.4 for all different traffic classes and encryption methods. The accuracy and average precision of the network are respectively 95.93% and 96.9% in the multi-class categorization

Table A.3

Complete performance results on one vs all categorization of different traffic classes. The task was to classify each traffic class versus all other classes with all the data from the test dataset.

Traffic class	Recall(%)	Precision(%)	Accuracy(%)
Browsing	99.3	99.0	99.4
Chat	96.4	94.3	99.4
File Transfer	98.9	100.0	99.9
Video	99.8	100.0	100.0
VoIP	99.6	99.9	99.8

Table A.4

Complete performance results on one vs all categorization of different traffic classes and encryption techniques. The task was to classify each class versus all other classes on the test dataset.

Traffic class	Recall(%)	Precision(%)	Accuracy(%)
Browsing-Unencrypted	99.2	99.5	99.6
Browsing-Tor	99.5	96.6	99.8
Chat-Unencrypted	92.5	88.6	99.6
Chat-Tor	92.1	89.7	99.8
Chat-VPN	100.0	99.2	100.0
File Transfer-Unencrypted	100.0	100.0	100.0
File Transfer-Tor	98.0	100.0	99.9
File Transfer-VPN	97.9	100.0	100.0
Video-Unencrypted	99.5	100.0	100.0
Video-Tor	100.0	100.0	100.0
Video-VPN	100.0	100.0	100.0
VoIP-Unencrypted	84.2	99.7	96.6
VoIP-Tor	98.3	100.0	99.9
VoIP-VPN	99.7	72.1	96.7

Table A.5

Average accuracies for on one vs all categorization of different traffic classes for each encryption technique. The reported results are the averages of results in Table 2 for each encryption technique. The corresponding Flowpic accuracies, from Table A.3 (rows 4–6) in [10], are always lower.

Traffic class	Our accuracy(%)	Flowpic's accuracy(%)
Unencrypted	99.7	97.0
Tor	99.4	85.7
VPN	99.9	99.7

of all types of traffic (traffic classes and encryption methods simultaneously with 14 output neurons). Finally, the average accuracies for one vs. all categorization of different traffic classes with the data of each encryption technique are reported in Table A.5.

References

- [1] L.L. Peterson, B.S. Davie, *Computer networks: a systems approach*, Elsevier, 2007.
- [2] R. Dingleline, N. Mathewson, P. Syverson, *Tor: The second-generation onion router*, Naval Research Lab Washington, DC, 2004, Tech. rep.
- [3] C. Scott, P. Wolfe, M. Erwin, *Virtual private networks*, O'Reilly Media Inc, 1999.
- [4] Y. LeCun, Y. Bengio, G. Hinton, Deep learning, *Nature* 521 (7553) (2015) 436–444, <https://doi.org/10.1038/nature14539>, URL: <http://www.nature.com/doi/10.1038/nature14539>.
- [5] J. Schmidhuber, Deep Learning in neural networks: An overview, *Neural Networks* 61 (2015) 85–117, <https://doi.org/10.1016/j.neunet.2014.09.003>, arXiv:1404.7828, URL: <https://doi.org/10.1016/j.neunet.2014.09.003>.
- [6] W. Maass, Networks of spiking neurons: The third generation of neural network models, *Neural Networks* 10 (9) (1997) 1659–1671, [https://doi.org/10.1016/S0893-6080\(97\)00011-7](https://doi.org/10.1016/S0893-6080(97)00011-7).
- [7] E.O. Neftci, H. Mostafa, F. Zenke, Surrogate Gradient Learning in Spiking Neural Networks: Bringing the Power of Gradient-based optimization to spiking neural networks, *IEEE Signal Process. Mag.* 36 (6) (2019) 51–63, <https://doi.org/10.1109/MSP.2019.2931595>.
- [8] Y. Wu, L. Deng, G. Li, J. Zhu, L. Shi, Spatio-temporal backpropagation for training high-performance spiking neural networks, *Front. Neurosci.* 12 (2018) 331.
- [9] S. Wozniak, A. Pantazi, T. Bohnstingl, E. Eleftheriou, Deep learning incorporating biologically inspired neural dynamics and in-memory computing, *Nature Mach. Intell.* 2 (6) (2020) 325–336.
- [10] T. Shapira, Y. Shavitt, FlowPic: Encrypted Internet Traffic Classification is as Easy as Image Recognition, *INFOCOM 2019 – IEEE Conference on Computer Communications Workshops, INFOCOM WKSHPS 2019* (2019) 680–687 doi:10.1109/INFOCOMW.2019.8845315.
- [11] A. Dainotti, A. Pescapé, K.C. Claffy, Issues and future directions in traffic classification, *IEEE Network* 26 (1) (2012) 35–40.
- [12] M. Crotti, M. Dusi, F. Gringoli, L. Salgarelli, Traffic classification through simple statistical fingerprinting, *ACM SIGCOMM Comput. Commun. Rev.* 37 (1) (2007) 5–16.
- [13] X. Wang, D.J. Parish, Optimised multi-stage tcp traffic classifier based on packet size distributions, in: 2010 Third International Conference on Communication Theory, Reliability, and Quality of Service, IEEE, 2010, pp. 98–103.
- [14] T. Qin, L. Wang, Z. Liu, X. Guan, Robust application identification methods for p2p and voip traffic classification in backbone networks, *Knowl.-Based Syst.* 82 (2015) 152–162.
- [15] A. Moore, D. Zuev, M. Crogan, Discriminators for use in flow-based classification, *Tech. rep.* (2013).
- [16] A.W. Moore, D. Zuev, Internet traffic classification using bayesian analysis techniques, in: Proceedings of the 2005 ACM SIGMETRICS international conference on Measurement and modeling of computer systems, 2005, pp. 50–60.
- [17] A. Fahad, Z. Tari, I. Khalil, I. Habib, H. Alnuweiri, Toward an efficient and scalable feature selection approach for internet traffic classification, *Comput. Netw.* 57 (9) (2013) 2040–2057.
- [18] T. Auld, A.W. Moore, S.F. Gull, Bayesian neural networks for internet traffic classification, *IEEE Trans. Neural Networks* 18 (1) (2007) 223–239.
- [19] G. Draper-Gil, A.H. Lashkari, M.S.I. Mamun, A.A. Ghorbani, Characterization of encrypted and VPN traffic using time-related features, *ICISSP 2016 – Proceedings of the 2nd International Conference on Information Systems Security and Privacy* (February) (2016) 407–414. doi:10.5220/0005740704070414.
- [20] B. Yamansavascular, M.A. Guvensan, A.G. Yavuz, M.E. Karsligil, Application identification via network traffic classification, in: 2017 International Conference on Computing, Networking and Communications (ICNC), IEEE, 2017, pp. 843–848.
- [21] J. Zhang, Y. Xiang, Y. Wang, W. Zhou, Y. Xiang, Y. Guan, Network traffic classification using correlation information, *IEEE Trans. Parallel Distrib. Syst.* 24 (1) (2012) 104–117.
- [22] J. Zhang, X. Chen, Y. Xiang, W. Zhou, J. Wu, Robust network traffic classification, *IEEE/ACM Trans. Networking* 23 (4) (2014) 1257–1270.
- [23] Z. Wang, The applications of deep learning on traffic identification, *BlackHat USA 24* (11) (2015) 1–10.
- [24] W. Wang, M. Zhu, J. Wang, X. Zeng, Z. Yang, End-to-end encrypted traffic classification with one-dimensional convolutional neural networks, in: 2017 IEEE International Conference on Intelligence and Security Informatics (ISI), IEEE, 2017, pp. 43–48.
- [25] M. Lotfollahi, M.J. Siavoshani, R.S.H. Zade, M. Saberian, Deep packet: A novel approach for encrypted traffic classification using deep learning, *Soft. Comput.* 24 (3) (2020) 1999–2012.
- [26] M. Lopez-Martin, B. Carro, A. Sanchez-Esguevilas, J. Lloret, Network Traffic Classifier with Convolutional and Recurrent Neural Networks for Internet of

- Things, IEEE Access 5 (September) (2017) 18042–18050, <https://doi.org/10.1109/ACCESS.2017.2747560>.
- [27] Z. Chen, K. He, J. Li, Y. Geng, Seq2img: A sequence-to-image based approach towards ip traffic classification using convolutional neural networks, in: 2017 IEEE International Conference on Big Data (Big Data), IEEE, 2017, pp. 1271–1276.
- [28] R. Sun, B. Yang, L. Peng, Z. Chen, L. Zhang, S. Jing, Traffic classification using probabilistic neural networks, 2010 Sixth International Conference on Natural Computation, vol. 4, IEEE, 2010, pp. 1914–1919.
- [29] M. Lotfollahi, R.S.H. Zade, M.J. Siavoshani, M. Saberian, Deep packet: A novel approach for encrypted traffic classification using deep learning (2018). arXiv:1709.02656.
- [30] A.H. Lashkari, G. Draper-Gil, M.S.I. Mamun, A.A. Ghorbani, Characterization of tor traffic using time based features, ICISp (2017) 253–262.
- [31] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning applied to document recognition, Proc. IEEE 86 (11) (1998) 2278–2324.
- [32] K. Roy, A. Jaiswal, P. Panda, Towards spike-based machine intelligence with neuromorphic computing, Nature 575 (7784) (2019) 607–617, <https://doi.org/10.1038/s41586-019-1677-2>.
- [33] R. Zimmer, T. Pellegrini, S.F. Singh, T. Masquelier, Technical report: supervised training of convolutional spiking neural networks with PyTorch, arXiv (2019) 1–24 arXiv:1911.10124. URL: <http://arxiv.org/abs/1911.10124>.
- [34] T. Pellegrini, R. Zimmer, T. Masquelier, Low-activity supervised convolutional spiking neural networks applied to speech commands recognition, in: IEEE Spoken Language Technology Workshop, 2021. arXiv:2011.06846. URL: <http://arxiv.org/abs/2011.06846>.
- [35] P. Soltani Zarrin, R. Zimmer, C. Wenger, T. Masquelier, Epileptic Seizure Detection Using a Neuromorphic-Compatible Deep Spiking Neural Network, in: Lecture Notes in Computer Science, vol. 12108, 2020, pp. 389–394. doi:10.1007/978-3-030-45385-5_34. URL: http://link.springer.com/10.1007/978-3-030-45385-5_34.
- [36] W. Fang, Z. Yu, Y. Chen, T. Masquelier, T. Huang, Y. Tian, Incorporating Learnable Membrane Time Constant to Enhance Learning of Spiking Neural Networks, arXiv arXiv:2007.05785. URL: <http://arxiv.org/abs/2007.05785>.
- [37] B. Yin, F. Corradi, S.M. Bohté, Effective and Efficient Computation with Multiple-timescale Spiking Recurrent Neural Networks, arXiv arXiv:2005.11633. URL: <http://arxiv.org/abs/2005.11633>.



Ali Rasteh received his B.S. and M.S. of Electrical Engineering-Electronics from Sharif University of Technology, Tehran, Iran, respectively, in 2015 and 2017. He is currently a Ph.D. student in Electrical Engineering-AI at Sharif University, cooperating with the CNRS (French National Science Foundation). His research focuses on Deep Neural Networks and, more specifically, Deep Spiking Networks. He is also interested in the intersection of machine learning and the applications of data networks.

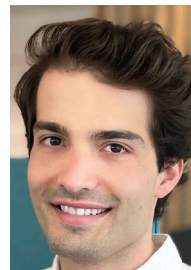


Florian Delpech is undertaking a Master Of Science in applied mathematics and computing at ISAE-SUPAERO, a leading school in aerospace engineering located in Toulouse, France. He is also graduated from Sciences Po Paris with a Master in Finance and Strategy. Florian has a strong passion for computational intelligence and more particularly for neural networks and evolutionary algorithms. He is convinced that their ability to solve difficult, combinatorial and multi-objective optimization problems will lead to significant innovations in a wide variety of disciplines such as robotics or biology and in numerous industries. He therefore believes that

Spiking Neural Networks have huge potential on embedded systems, such as satellites, to solve complex resource allocation problems with low energy footprint hardware.



Former student of l'École Polytechnique de Paris, Carlos Aguilar Melchor obtained his Ph.D. degree in 2006 at LAAS/CNRS (Toulouse) under the supervision of Yves Deswarthe. His research domains are security, privacy, cryptography and artificial intelligence. In security and privacy, he has worked mainly on anonymous communications and on trust and replication issues on new generation networks. In cryptography, he has worked on various primitives such as private information retrieval, homomorphic encryption, ring signatures and public key encryption, with a special focus on the post-quantum setting. In artificial intelligence, he has worked on anomaly detection and on the robustness and transfer properties of reinforcement learning algorithms. He has published 45 research papers on international journals and conferences and has been for seven years member of the national university council, head of the network and infrastructure security work-group of the CNRS (french national science foundation) and member of the bureau for cybersecurity issues of the CNRS. He is currently full professor at ISAE SUPAERO, in Toulouse, France.



Romain Zimmer is a machine learning researcher and engineer interested in efficient neural information processing. He developed s2net, a Spiking Neural Networks (SNNs) python library based on PyTorch to train SNNs with surrogate gradient methods. He is also interested in new artificial intelligence approaches combining knowledge based systems and machine learning. Romain is currently working on Sievable, a new filter-based neural search engine that allows you to filter the web according to your own needs.



Saeed Bagheri Shouraki received his BSc and MSc in Electrical Engineering (Electronics and Digital Systems, respectively) from Sharif University of Technology, Tehran, Iran. He received his PhD in Electrical Engineering (Control-Fuzzy Systems and Soft Computing) from University of Electro-communications, Tokyo, Japan. He joined Sharif University of Technology as a faculty member of Computer Engineering Department in 1987 and later transferred to Department of Electrical Engineering in 2007 as a Professor. His research interests cover a variety of subjects including Analog and Digital Hardware Co-design, Robotics, Machine Learning, Soft Computing, Fuzzy Systems and Neural Networks, and Artificial Intelligence. Developing state-of-the-art processing systems based on novel operators in order to develop intelligent hardware, and their implementation by both memristors and other nano-scaled devices remains at the center of his researches. He is the head of ACL (Artificial Creatures Lab), former chair of Computer Engineering Department, and current chair of Electrical Engineering Department, Sharif University of Technology.



Timothée Masquelier is a CNRS senior research scientist in computational neuroscience and AI. His research is highly interdisciplinary - at the interface between biology, computer science, and physics. He uses numerical simulations and analytical calculations to gain understanding on how the brain works, and more specifically on how neurons process, encode and transmit information through action potentials (a.k.a spikes), in particular in the visual and auditory modalities. He is also interested in bio-inspired computer vision and audition. He is convinced that spiking neural networks (SNN) can be an appealing alternative to the conventional artificial neural networks commonly used in AI, especially if implemented on low-power neuromorphic chips.